1. Truffle Installation Steps

What is Truffle?

Truffle Suite is a framework used to:

- Write smart contracts
- Compile contracts
- Deploy contracts
- Test blockchain applications

It works mainly with Ethereum blockchain.

Why do we need Truffle?

Without Truffle:

- We manually compile contracts
- Manually deploy them
- Difficult testing

With Truffle:

- Everything becomes automated

Step-by-Step Installation

Step 1: Install Node.js

Why?

Truffle runs using Node.js.

Steps

1. Open browser
2. Go to:
   Node.js Official Website
3. Download LTS version

4. Install normally

Step 2: Verify Installation

Open terminal or command prompt.

Type:

node -v

What happens?

- System checks Node.js version
- Example output:

v20.10.0

Now check npm:

npm -v

npm = Node Package Manager.

Step 3: Install Truffle

Command:

npm install -g truffle

Explanation

- `npm` → package installer
- `install` → install package
- `-g` → global installation
- `truffle` → package name

What happens internally?

npm downloads:

- Truffle libraries
- Solidity compiler support

- Deployment tools

Step 4: Verify Truffle

truffle version

Example:

Truffle v5.x.x

Viva Questions

Q1. What is Truffle?

Truffle is a blockchain development framework used for developing Ethereum smart contracts.

Q2. Why Node.js is required?

Because Truffle runs on JavaScript runtime Node.js.

Q3. What is npm?

npm is Node Package Manager used to install packages.

---

2. Simple Solidity Program and Deployment on Truffle Local Blockchain

---

What is Solidity?

Solidity is a programming language used to create Ethereum smart contracts.

What is Smart Contract?

A smart contract is:

- Self-executing code

- Stored on blockchain
- Runs automatically

Example:
If payment received → transfer ownership automatically.

Step-by-Step Program

Step 1: Create Truffle Project

mkdir MyProject
cd MyProject
truffle init

Explanation

- `mkdir` → creates folder
- `cd` → enter folder
- `truffle init` → creates project structure

Step 2: Understand Project Structure

Folders created:

contracts/

Contains Solidity files

migrations/

Deployment scripts

test/

Testing files

Step 3: Create Solidity File

Inside `contracts` folder:

Create `SimpleStorage.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {

   uint public data;

   function setData(uint _data) public {
      data = _data;
   }

   function getData() public view returns(uint) {
      return data;
   }
}
```

Line-by-Line Explanation

SPDX License

// SPDX-License-Identifier: MIT

Defines software license.

Solidity Version

pragma solidity ^0.8.0;

Tells compiler version.

Contract Declaration

contract SimpleStorage

Creates smart contract.

State Variable

uint public data;

- `uint` = unsigned integer
- Stored permanently on blockchain

Function to Store Data

function setData(uint _data) public

Stores value.

Function to Retrieve Data

function getData() public view returns(uint)

Returns stored value.

Step 4: Compile Contract

truffle compile

What happens?

- Solidity code converted into bytecode
- ABI generated

Step 5: Install Ganache

Ganache creates local blockchain for testing.

Download:
Ganache Official Website

Step 6: Start Ganache

Open Ganache.

It creates:

- Local accounts
- Fake ETH
- Local blockchain

Step 7: Create Migration File

Inside migrations folder:

`2_deploy.js`

```
const SimpleStorage = artifacts.require("SimpleStorage");

module.exports = function(deployer) {
  deployer.deploy(SimpleStorage);
};
```

Step 8: Deploy Contract

truffle migrate

What happens internally?

1. Truffle connects Ganache
2. Sends contract
3. Blockchain stores contract
4. Contract address generated

Viva Questions

Q1. What is blockchain?

Blockchain is decentralized immutable digital ledger.

Q2. What is Ganache?

Ganache is local Ethereum blockchain simulator.

Q3. What is smart contract deployment?

Uploading smart contract to blockchain.

---

3. MetaMask Installation

What is MetaMask?

MetaMask is:

- Crypto wallet
- Ethereum wallet
- Connects browser with blockchain

Step-by-Step Installation

Step 1: Open Website

MetaMask Official Website

Step 2: Install Extension

Choose browser:

- Chrome
- Edge
- Firefox

Step 3: Create Wallet

Click:

- Create Wallet

Step 4: Create Password

Used to access wallet locally.

Step 5: Save Secret Recovery Phrase

Very important.

Why?

Used to recover wallet.

Never share it.

What happens internally?

MetaMask:

- Generates public/private keys
- Creates Ethereum address

Viva Questions

Q1. What is MetaMask?

MetaMask is Ethereum wallet and Web3 gateway.

Q2. What is private key?

Secret key used to control wallet.

Q3. Why recovery phrase is important?

Used to restore wallet.

---

4. Smart Contract using MetaMask and Remix

What is Remix?

Remix IDE is online IDE for Solidity.

Website:
Remix IDE

Step-by-Step

Step 1: Open Remix

Open browser.

Step 2: Create Solidity File

Create `Hello.sol`

```
pragma solidity ^0.8.0;

contract Hello {
   string public message = "Hello Blockchain";
}
```

Step 3: Compile

Click Solidity Compiler.

Click Compile.

Step 4: Connect MetaMask

In Remix:

- Deploy & Run Transactions
- Environment → Injected Provider MetaMask

MetaMask popup appears.

Connect wallet.

Step 5: Deploy Contract

Click Deploy.

MetaMask asks confirmation.

Pay gas fee.

Contract deployed.

What is Gas Fee?

Gas = transaction fee in Ethereum.

Used for:

- Computation
- Storage

Viva Questions

Q1. What is Remix?

Browser IDE for Solidity.

Q2. What is gas?

Fee for executing transaction.

Q3. Why MetaMask used?

To sign blockchain transactions.

---

5. Web3 Program

What is Web3?

Web3.js connects:

- Website
- Blockchain

Simple Web3 Example

Install:

npm install web3

Program

```
const Web3 = require('web3');

const web3 = new Web3('http://127.0.0.1:7545');

web3.eth.getAccounts().then(console.log);
```

Explanation

require('web3')

Imports library.

---

new Web3()

Connects blockchain.

---

getAccounts()

Fetches Ethereum accounts.

Viva Questions

Q1. What is Web3?

JavaScript library to interact with Ethereum.

Q2. Why Web3 used?

To connect frontend with blockchain.

---

6. Study of Cryptocurrency Development

What is Cryptocurrency?

Digital currency using cryptography.

Examples:

- Bitcoin
- Ethereum

Features

- Decentralized
- Secure
- Transparent
- Immutable

Cryptocurrency Development Steps

1. Define blockchain
2. Create consensus mechanism
3. Develop wallet
4. Add mining/staking
5. Launch network

Viva Questions

Q1. What is cryptocurrency?

Digital decentralized currency.

Q2. What is mining?

Process of validating transactions.

---

7. Chaincode in Hyperledger Fabric

What is Hyperledger Fabric?

Hyperledger Fabric is permissioned blockchain.

Developed by:
Linux Foundation

What is Chaincode?

Chaincode = smart contract in Hyperledger.

Features

- Permissioned network
- Faster
- Enterprise use

Basic Steps

1. Install Docker
2. Install Hyperledger Fabric
3. Create network
4. Write chaincode
5. Deploy chaincode

Viva Questions

Q1. Difference between Ethereum and Hyperledger?

Ethereum	Hyperledger
Public	Private
Cryptocurrency	Enterprise
Ether required	No cryptocurrency

8.Decentralized Voting System

Problem Statement: Design and implement a smart contract in Remix that allows users to vote for candidates in a secure and transparent election. The system should ensure one vote per address and allow the admin to start and end the voting period.

‹/› solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Voting {

    // Candidate structure
    struct Candidate {
        string name;
        uint voteCount;
    }

    // Store candidates
    Candidate[] public candidates;

    // Track voted users
    mapping(address => bool) public voted;

    // Constructor adds candidates
    constructor() {
        candidates.push(Candidate("Alice", 0));
        candidates.push(Candidate("Bob", 0));
    }

    // Voting function
    function vote(uint candidateIndex) public {

        // Check if already voted
        require(!voted[msg.sender], "You already voted");

        // Mark as voted
        voted[msg.sender] = true;

        // Increase vote count
        candidates[candidateIndex].voteCount++;
    }

    // Get total candidates
    function getCandidates() public view returns(uint) {
        return candidates.length;
    }
}
```

Compile Code

In Remix:

1. Click Solidity Compiler
2. Click Compile Voting.sol

Deploy Contract

Go to:

- Deploy & Run Transactions

Select:

JavaScript VM

OR use MetaMask later.

Click Deploy

Contract deploys.

Test Voting

Check candidates

Call:

candidates(0)

Shows Alice.

Vote

In vote function enter:

0

Means vote for Alice.

Click transact.

Check Vote Count

Again call:

candidates(0)

Simple Viva Questions and Answers

Q1. What is decentralized voting?

Voting system stored on blockchain.

Q2. Why blockchain used?

For security and transparency.

Q3. What is mapping?

Key-value data structure.

Q4. What is msg.sender?

Current wallet address.

Q5. Why require() used?

To check conditions and stop invalid transactions.

Q6. How duplicate voting prevented?

Using mapping(address => bool).

Q7. What is constructor?

Function that runs once during deployment.

9.Crowdfunding Platform

Problem Statement: Create a smart contract that allows users to create fundraising campaigns and accept donations. If the funding goal is not met by the deadline, donors should be able to withdraw their funds.

solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Crowdfunding {

    address public owner;
    uint public goal;
    uint public totalFunds;

    mapping(address => uint) public donations;

    constructor(uint _goal) {
        owner = msg.sender;
        goal = _goal;
    }

    // Donate ETH
    function donate() public payable {
        donations[msg.sender] += msg.value;
        totalFunds += msg.value;
    }

    // Withdraw funds
    function withdraw() public {

        require(msg.sender == owner, "Only owner can withdraw");

        require(totalFunds >= goal, "Goal not reached");

        payable(owner).transfer(totalFunds);
    }

    // Refund money
    function refund() public {

        require(totalFunds < goal, "Goal reached");

        uint amount = donations[msg.sender];

        donations[msg.sender] = 0;

        payable(msg.sender).transfer(amount);
    }
}
```

How to Run

Compile

Go to:

- Solidity Compiler
- Compile Crowdfunding.sol

Deploy

Go to:

- Deploy & Run Transactions

Constructor input example:

5

Means target = 5 ETH.

Donate

Before clicking donate():

- Enter Value = 1 ETH

Then click:

donate

Viva Questions

Q1. What is payable?

Function that accepts ETH.

Q2. What is msg.sender?

Current user address.

Q3. What is msg.value?

Amount of ETH sent.

Q4. Why mapping used?

To store user donations.

10.Decentralized To-Do List
Problem Statement:
Develop a smart contract that allows users to add, update, and mark tasks as completed. Each user's tasks should be stored independently and be accessible only by the user.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Todo {

    // Task structure
    struct Task {
        string task;
        bool completed;
    }

    // Store tasks of each user
    mapping(address => Task[]) public tasks;

    // Add new task
    function addTask(string memory _task) public {

        tasks[msg.sender].push(Task(_task, false));
    }

    // Mark task as completed
    function completeTask(uint index) public {

        tasks[msg.sender][index].completed = true;
    }

    // Update task
    function updateTask(uint index, string memory newTask) public {

        tasks[msg.sender][index].task = newTask;
    }
}
```

11.Simple Bank (Wallet System)

Problem Statement: Build a smart contract that functions as a simple bank. Users should be able to deposit, withdraw, and check their balance. Only the account holder should have access to their funds.

solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleBank {

    // Store balance of each user
    mapping(address => uint) public balances;

    // Deposit ETH
    function deposit() public payable {

        balances[msg.sender] += msg.value;
    }

    // Withdraw ETH
    function withdraw(uint amount) public {

        require(balances[msg.sender] >= amount, "Insufficient balance");

        balances[msg.sender] -= amount;

        payable(msg.sender).transfer(amount);
    }

    // Check balance
    function checkBalance() public view returns(uint) {

        return balances[msg.sender];
    }
}
```

Viva Questions and Answers

Q1. What is payable?

Allows contract/function to receive ETH.

Q2. What is msg.sender?

Current user wallet address.

Q3. What is msg.value?

Amount of ETH sent.

Q4. What is transfer()?

Used to send ETH.

Q5. Why mapping used?

To store balance of each user.

Q6. What is require()?

Checks condition before execution.

Q7. Why view keyword used?

Function only reads blockchain data.

12.Supply Chain Management

Problem Statement: Create a smart contract to track products from manufacturer to consumer. Each step in the supply chain should be recorded immutably with timestamps and participant addresses.

‹/› solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SupplyChain {

    // Product structure
    struct Product {
        string name;
        string status;
        address owner;
    }

    // Store products
    mapping(uint => Product) public products;

    // Add product
    function addProduct(
        uint productId,
        string memory name,
        string memory status
    ) public {

        products[productId] = Product(
            name,
            status,
            msg.sender
        );
    }

    // Update product status
    function updateStatus(
        uint productId,
        string memory newStatus
    ) public {

        products[productId].status = newStatus;
    }
}
```

Q1. What is supply chain management?

Tracking products from manufacturer to consumer.

Q2. What is immutable history?

Stored data cannot be modified.

Q3. Why blockchain used in supply chain?

For transparency and security.

Q4. What is struct?

Custom datatype storing related data.

Q5. What is mapping?

Key-value storage structure.

Q6. What is msg.sender?

Current user wallet address.

Q7. Why owner address stored?

To track product ownership.

13.NFT Minting Contract

Problem Statement: Develop a smart contract that allows users to mint and transfer non-fungible tokens (NFTs) representing digital assets. Include metadata for each NFT and ensure ownership tracking.

solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract MyNFT {

    // NFT structure
    struct NFT {
        string name;
        address owner;
    }

    // Store NFTs
    mapping(uint => NFT) public nfts;

    // Mint NFT
    function mintNFT(uint tokenId, string memory name) public {

        nfts[tokenId] = NFT(name, msg.sender);
    }

    // Transfer NFT ownership
    function transferNFT(uint tokenId, address newOwner) public {

        require(
            nfts[tokenId].owner == msg.sender,
            "You are not the owner"
        );

        nfts[tokenId].owner = newOwner;
    }
}
```

STEP 1: Compile

Go to:

- Solidity Compiler
- Click:

Compile MyNFT.sol

STEP 2: Deploy

Go to:

- Deploy & Run Transactions

Select:

JavaScript VM

Click:

Deploy

STEP 3: Mint NFT

Find:

mintNFT

Enter:

Field	Value
tokenId	1
name	MyArt

Viva Questions and Answers

Q1. What is NFT?

Unique digital asset stored on blockchain.

Q2. What is minting?

Creating NFT on blockchain.

Q3. What is ERC-721?

Ethereum NFT standard.

Q4. Why NFTs unique?

Each NFT has unique token ID and owner.

Q5. What is msg.sender?

Current user wallet address.

Q6. Why require() used?

To verify ownership before transfer.

Q7. What is metadata?

Extra NFT information like image or description.

14.Subscription Service Problem

Statement: Implement a smart contract that manages subscription-based access to a service. Users should pay to subscribe for a period of time, and access should automatically expire afterward.

solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Subscription {

    // Store expiry time of users
    mapping(address => uint) public expiry;

    // Subscription fee
    uint public fee = 1 ether;

    // Subscribe function
    function subscribe() public payable {

        // Check payment amount
        require(msg.value >= fee, "Insufficient payment");

        // Add 30 days subscription
        expiry[msg.sender] = block.timestamp + 30 days;
    }

    // Check subscription status
    function isSubscribed() public view returns(bool) {

        // True if current time is less than expiry
        return block.timestamp < expiry[msg.sender];
    }
}
```

Step 1: Compile

Click:

- Solidity Compiler
- Compile Subscription.sol

If green tick appears:

- code correct

Step 2: Deploy

Go to:

- Deploy & Run Transactions

Choose:

- Remix VM

Click:

- Deploy

Step 3: Subscribe

In VALUE box:
Enter:

1

ETH

Then click:

Subscribe

Simple Viva Questions and Answers

Q1. What is subscription smart contract?

Blockchain program for paid access system.

Q2. Why mapping used?

To store user expiry times.

Q3. What is payable?

Allows function to receive ETH.

Q4. What is msg.value?

Amount of ETH sent by user.

Q5. What is block.timestamp?

Current blockchain time.

Q6. How subscription expiry checked?

By comparing current time with stored expiry time.

Q7. Why blockchain used here?

For secure and automatic subscription management.

15.Escrow Payment System

Problem Statement: Design a smart contract that acts as an escrow between a buyer and a seller. Funds should be released only when both parties agree or after a certain time if no disputes are raised.

‹/› solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Escrow {

    // Buyer address
    address public buyer;

    // Seller address
    address payable public seller;

    // Amount deposited
    uint public amount;

    // Constructor
    constructor(address payable _seller) {
        buyer = msg.sender;
        seller = _seller;
    }

    // Deposit money into contract
    function deposit() public payable {

        // Only buyer can deposit
        require(msg.sender == buyer, "Only buyer allowed");

        amount = msg.value;
    }

    // Buyer confirms product received
    function confirmDelivery() public {

        // Only buyer can confirm
        require(msg.sender == buyer, "Only buyer allowed");

        // Transfer money to seller
        seller.transfer(amount);
    }
}
```

Step 1: Compile

Click:

- Solidity Compiler
- Compile Escrow.sol

Green tick means:

- code correct

Step 2: Deploy

Go to:

- Deploy & Run Transactions

Choose:

- Remix VM

Step 3: Enter Seller Address

In deploy input:
paste one Remix account address.

Example:

0xAbc123...

This becomes seller.

Step 4: Deploy

Click:

- Deploy

Simple Viva Questions and Answers

Q1. What is escrow?

Third-party system that holds money until conditions are satisfied.

Q2. Why blockchain escrow secure?

Funds controlled automatically by smart contract.

Q3. What is payable?

Allows address/function to receive ETH.

Q4. What is msg.sender?

Current wallet address.

Q5. What is msg.value?

Amount of ETH sent in transaction.

Q6. Why require() used?

To check conditions and stop invalid actions.

Q7. What does transfer() do?

Transfers ETH from contract to seller.

16.Decentralized Lottery System

Problem Statement: Create a lottery smart contract where users can buy tickets, and a random winner is picked at the end of the round. Ensure fairness and prevent manipulation.

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Lottery {

    address payable[] public players;

    // Enter lottery
    function enter() public payable {
        require(msg.value == 1 ether);

        players.push(payable(msg.sender));
    }

    // Pick winner
    function pickWinner() public {

        uint random = uint(
            keccak256(
                abi.encodePacked(block.timestamp, players.length)
            )
        );

        uint index = random % players.length;

        players[index].transfer(address(this).balance);
    }
}
```

How To Run

Step 1

Compile `Lottery.sol`

Step 2

Deploy contract

Step 3

Enter lottery:

- put 1 ETH in VALUE box
- click `enter`

Step 4

Use different Remix accounts and repeat.

Step 5

Click:

pickWinner

Winner receives all ETH.

Simple Viva Questions

Q1. What is decentralized lottery?

Blockchain-based lottery system.

Q2. Why payable used?

To receive/send ETH.

Q3. What is msg.sender?

Current user address.

Q4. What is msg.value?

ETH sent by user.

Q5. What is keccak256?

Ethereum hash function.

17.Certificate Issuance and Verification

Problem Statement: Build a smart contract that issues digital certificates (e.g., for a course or event) and allows anyone to verify their authenticity using the certificate ID or address.

‹/› solidity

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CertificateSystem {

    // Certificate structure
    struct Certificate {
        string name;
        string course;
        bool valid;
    }

    // Store certificates
    mapping(uint => Certificate) public certificates;

    // Add certificate
    function issueCertificate(
        uint id,
        string memory name,
        string memory course
    ) public {

        certificates[id] = Certificate(name, course, true);
    }

    // Verify certificate
    function verifyCertificate(uint id)
        public
        view
        returns(bool)
    {
        return certificates[id].valid;
    }
}
```

How To Run in Remix

Step 1: Compile

Click:

- Solidity Compiler
- Compile Certificate.sol

Green tick means:

- code correct

Step 2: Deploy

Go to:

- Deploy & Run Transactions

Choose:

- Remix VM

Click:

- Deploy

Step 3: Issue Certificate

Enter:

- id = 1
- name = Rahul
- course = Blockchain

Click:

issueCertificate

Step 4: Verify Certificate

Enter:

1

Click:

verifyCertificate

Returns:

true

Meaning certificate is valid.

Simple Viva Questions and Answers

Q1. What is blockchain certificate system?

System for storing and verifying certificates using blockchain.

Q2. Why blockchain certificates secure?

Because blockchain data cannot be changed.

Q3. What is immutable?

Data cannot be modified after storing.

Q4. What is mapping?

Key-value data structure.

Q5. What is bool?

Boolean value:

- true
- false

Q6. Why certificate verification important?

To prevent fake certificates.